Inne sposoby poprawy wydajności

1. Audyt istniejącej struktury (analiza statystyk i struktury bazodanowej)
2. Faktycznie wykorzystanie obiektów (Indeksy / Triggers / Foreign Keys – czy potrzebne?)
3. Skalowanie bazy danych (dokładanie zasobów / tworzenie klastra)
4. Zmiana użytych dysków dla bazy danych (Operacje I/O)
5. Zmiana parametrów konfiguracyjnych (**postgresql.conf**)
6. Wersja używanej bazy danych?
7. Wersja wykorzystywanych sterowników (ODBC / JDBC)?
8. Wpływ konfiguracji sieciowej na przepływ danych? (Onsite -> Cloud ? / VPN?)

https://wiki.postgresql.org/wiki/Performance_Optimization

https://wiki.postgresql.org/wiki/Slow_Query_Questions

```
#------------------------------------------------------------------------------
# RESOURCE USAGE (except WAL)
#------------------------------------------------------------------------------

# - Memory -

shared_buffers = 128MB                  # min 128kB
                                        # (change requires restart)
#huge_pages = try                       # on, off, or try
                                        # (change requires restart)
#temp_buffers = 8MB                     # min 800kB
#max_prepared_transactions = 0          # zero disables the feature
                                        # (change requires restart)
# Caution: it is not advisable to set max_prepared_transactions nonzero unless
# you actively intend to use prepared transactions.
#work_mem = 4MB                         # min 64kB
#maintenance_work_mem = 64MB            # min 1MB
#autovacuum_work_mem = -1               # min 1MB, or -1 to use maintenance_work_mem
#max_stack_depth = 2MB                  # min 100kB

                        #------------------------------------------------------------------------------
                        # WRITE-AHEAD LOG
                        #------------------------------------------------------------------------------

                        # - Settings -

                        #wal_level = replica                    # minimal, replica, or logical
                                                                # (change requires restart)
                        #fsync = on                             # flush data to disk for crash safety
                                                                # (turning this off can cause
                                                                # unrecoverable data corruption)
                        #synchronous_commit = on                # synchronization level;
                                                                # off, local, remote_write, remote_apply, or on
                        #wal_sync_method = fsync                # the default is the first option
                                                                # supported by the operating system:
                                                                #   open_datasync
                                                                #   fdatasync (default on Linux)
                                                                #   fsync
                                                                #   fsync_writethrough
                                                                #   open_sync
                        #full_page_writes = on                  # recover from partial page writes
                        #wal_compression = off                  # enable compression of full-page writes
```

```
#------------------------------------------------------------------------------
# QUERY TUNING
#------------------------------------------------------------------------------

# - Planner Method Configuration -

#enable_bitmapscan = on
#enable_hashagg = on
#enable_hashjoin = on
#enable_indexscan = on
#enable_indexonlyscan = on
#enable_material = on
#enable_mergejoin = on
#enable_nestloop = on
#enable_parallel_append = on
#enable_seqscan = on
#enable_sort = on
#enable_tidscan = on
#enable_partitionwise_join = off
#enable_partitionwise_aggregate = off
#enable_parallel_hash = on
#enable_partition_pruning = on

# - Planner Cost Constants -

#seq_page_cost = 1.0                    # measured on an arbitrary scale
#random_page_cost = 4.0                 # same scale as above
#cpu_tuple_cost = 0.01                  # same scale as above
#cpu_index_tuple_cost = 0.005           # same scale as above
#cpu_operator_cost = 0.0025             # same scale as above
#parallel_tuple_cost = 0.1              # same scale as above
#parallel_setup_cost = 1000.0   # same scale as above

#jit_above_cost = 100000                # perform JIT compilation if available
                                        # and query more expensive than this;
                                        # -1 disables
#jit_inline_above_cost = 500000         # inline small functions if query is
                                        # more expensive than this; -1 disables
#jit_optimize_above_cost = 500000       # use expensive JIT optimizations if
                                        # query is more expensive than this;
                                        # -1 disables
```